REPORT — Ephemeral Secret Vault

> **It Happens @ RAALE #9** · A self-destructing, zero-trace secret-sharing service
> **Stack:** Python 3.11 · FastAPI · SQLite (WAL) · AES-256-GCM · HTML/CSS/JS
> **Measured on:** Windows 11 laptop (AMD Ryzen 5 5600H), 2026-09-24. Every number here was measured and can be reproduced with the commands in §7.

Read this first (60-second summary)

The problem. People share passwords and API keys in Slack, Teams and email. Those messages are kept forever, so one hacked account later exposes every secret ever pasted there.

What we built. A service that turns a secret into a **one-time link**:

1. You paste a secret, or pipe it in from the terminal. It is **encrypted** immediately, and the server's database never holds the readable secret.
2. You get a short link to share.
3. The recipient opens the link and clicks **"Reveal and Destroy Secret"**. They see the secret **once**, and it is then **permanently deleted**.
4. If nobody opens it before its timer runs out, it is deleted automatically.

The four hard guarantees, all proven by automated tests:

Guarantee	In plain English	Result
🔐 Encrypted at rest	Stealing the database file gives an attacker gibberish, not secrets.	✅ 0 plaintext bytes found on disk
🤖 Link-preview safe	Slack or WhatsApp "previewing" the link can't use it up.	✅ 12 bot types tested; none consumed a secret
⚡ Only one reader	If 20 people click at the exact same millisecond, exactly one gets the secret.	✅ 1 winner, 19 refused, in 10 of 10 test rounds
🧹 Self-cleaning	Expired, unread secrets are deleted by a background cleaner.	✅ physically deleted within 2 cleaner cycles

Scoreboard: all **10/10 mandatory requirements** met · **206/206 automated tests** passing · **4/4 optional stretch goals** built · performance target met with 4 workers (§5.5).

Contents

- Scorecard: requirements → evidence
- 1. Architecture Overview
- 2. Security & Threat Model
- 3. The Scraper Problem
- 4. Concurrency Strategy

- 5. Results & Benchmarks
- 6. Limitations & Next Steps (honest account)
- 7. Quickstart
- Glossary

Scorecard: requirements → evidence

Mandatory requirements (handout Part 7.1)

#	Requirement	How it's done	Proof
1	AES-256-GCM at rest; a DB leak reveals no plaintext	Every secret is encrypted before it's stored; the key lives only in an environment variable	Disk scan: **0** plaintext matches in `vault.db` + WAL (`scripts/inspect_db.sh`, `test_no_plaintext_on_disk`)
2	A unique random IV per secret	`os.urandom(12)` for every secret	1,000 encryptions → 1,000 distinct IVs; DB: distinct IVs = rows
3	Short, non-guessable, URL-safe ID	16 hex characters from `secrets.token_hex(8)` (64 random bits)	`test_ids`, `test_invalid_ids`
4	`GET /view/:id` shows HTML and doesn't burn	Read-only `SELECT`; decryption never happens on GET	Page viewed 10× → views unchanged (`test_view_page_is_read_only`)
5	Burn only via `POST /api/secret/:id/burn`	Only that route decrypts and deletes; `GET` on it → 405	`test_get_on_burn_405`, browser flow
6	A 1-view secret can't be read twice under concurrency	One atomic SQL statement inside a locked transaction (§4)	20 parallel requests → **1×200, 19×404, row deleted**, 10/10 rounds
7	Expired secrets unreadable and auto-deleted	Reads check expiry; background sweeper every 10 s	Live test: row physically gone within 2 cycles
8	Tampered data fails cleanly, no stack traces	GCM authentication fails → row deleted → generic 404	8/8 tamper cases pass; no `Traceback` in any response
9	Correct status codes (201 / 200 / 404)	Exact API contract from the handout	`test_happy_path` and 30 other API tests
10	CLI / curl pushes a secret from stdin	`cat secret.txt | ./cli/vault-cli` prints the link	`test_cli_pipeline`; verified in Git Bash

Marking criteria (handout Part 11) → where to look

Component (marks)	Evidence in this report
Cryptographic rigor (20)	§2.2 primitives, §2.3 key and IV management, §5.2 disk scan
Concurrency defense (20)	§4 strategy, §4.3 mutation test, §5.1 race results
Scraper shield (15)	§3 layers, §5.2 bot tests
Garbage collection (15)	§1.3 lifecycle, §5.1 TTL cleanup, §2.4 deletion guarantee
API quality & ergonomics (10)	§1.4 API contract, §7 quickstart, CLI

Component (marks)	Evidence in this report
Hostile input resilience (10)	§2.5 defenses, §5.2 tamper and hostile-input tests
Report: architecture, honesty, results (10)	This document, especially §6 (what isn't perfect, and the bugs we found)

---

1. Architecture Overview

> **In plain English:** a small web server with three internal parts. One **locks** secrets (encryption), one **stores** them (a database file), and one **cleans up** expired ones. A web page and a terminal tool are the two ways in.

1.1 The big picture



1.2 Components

Component	File	Job
Gate middleware	`app/main.py`	Blocks bots, adds security headers, turns crashes into generic errors
API and page routes	`app/routes/api.py`, `pages.py`	The HTTP contract
VaultService	`app/service.py`	Business rules: create, burn, expiry, password check
CryptoEngine	`app/crypto.py`	AES-256-GCM encrypt/decrypt only
SecretRepository	`app/repository.py`	Every SQL statement, including the atomic burn

Component	File	Job
Sweeper	`app/sweeper.py`	Background task: deletes expired rows every 10 s
Web UI	`app/templates`, `app/static`	Create page and landing page (plain HTML/CSS/JS, strict CSP)
CLI	`cli/vault-cli`	`cat secret.txt | ./vault-cli` → link; no extra packages needed

1.3 Life of a secret

```
create ──► ACTIVE (encrypted row) ──► reveal (last view) ──► DELETED (row gone)
              │                  └──► timer expires ──────► DELETED by sweeper
              │                  └──► data tampered ──────► DELETED, reader gets 404
              └── viewing the landing page or a bot visit never changes anything
```

There is no "soft delete": a destroyed secret's row is **physically removed**, not flagged.

1.4 API contract (exactly as specified in the handout)

Request	Success	Failure
`POST /api/secret` `{"secret","ttl_seconds","max_views"}`	**201** `{id, view_url, expires_at, views_remaining}`	400 invalid input
`GET /view/{id}`	**200** HTML: notice, expiry, views left, **Reveal and Destroy Secret** button	404 HTML page
`POST /api/secret/{id}/burn`	**200** `{secret, views_remaining, burned}`	**404** `{"error": "Secret not found, expired, or already destroyed."}`

1.5 Database schema (the handout's single table, unchanged)

```
CREATE TABLE secrets (
  id TEXT PRIMARY KEY,           -- 16 random hex chars
  ciphertext BLOB NOT NULL,      -- encrypted secret (never plaintext)
  iv BLOB NOT NULL,              -- 12 random bytes, unique per secret
  auth_tag BLOB NOT NULL,        -- 16-byte integrity tag
  max_views INTEGER NOT NULL DEFAULT 1,
  views_remaining INTEGER NOT NULL DEFAULT 1,
  expires_at INTEGER NOT NULL,   -- epoch ms
  created_at INTEGER NOT NULL
);
CREATE INDEX idx_secrets_expiry ON secrets(expires_at);
```

There's no plaintext column, key column or `is_deleted` flag. The optional stretch features add four columns through an automatic in-place migration: `pw_salt` and `pw_hash` (nullable), and `pw_failures` and `e2e` (default 0). The core table above stays exactly as the handout defines it.

---

2. Security & Threat Model

> **In plain English:** every secret is locked with a strong, modern lock (AES-256-GCM) before it's stored. The key is kept apart from the database, so stealing the database file gives an attacker only scrambled data. The lock also detects tampering: if anyone edits the stored data, it refuses to open instead of returning corrupted text.

2.1 What we protect against

Threat	Our defence	Leftover risk
Database file stolen	AES-256-GCM; the key is only in an environment variable	Attacker needs **both** the DB and the key (end-to-end mode, §5.4, removes even that)
Stored data edited or truncated	GCM integrity tag + row ID bound in as extra authenticated data → decryption refuses; row deleted; 404	None
Ciphertext copied from one row to another	Row ID is part of the authenticated data → decryption fails	None
Chat bots consuming the link	GET never burns; bots blocked (§3)	A bot that fakes a browser *and* sends a deliberate POST (unfurlers don't)
Two readers at once	Atomic database operation (§4)	None
Guessing IDs	64-bit random IDs; every miss looks identical (404)	No rate limiting yet (§6)
Error pages leaking internals	Generic JSON/HTML errors; logs record only error *types*	None
Secret shown as HTML/JS (XSS)	Displayed with `textContent`; strict Content Security Policy	None
Link leaking via caches or referrers	`Cache-Control: no-store`, `Referrer-Policy: no-referrer`, `noindex`	Browser history keeps the URL (harmless once burned)

2.2 Crypto primitives

Item	Choice	Why
Cipher	**AES-256-GCM** (`cryptography` library)	Authenticated encryption: secrecy and tamper detection in one step
IV (nonce)	**12 random bytes per secret** (`os.urandom`)	Reusing an IV with GCM breaks it; a fresh random IV every time prevents that
Auth tag	**16 bytes**, stored in its own column	Any change to the ciphertext, IV or tag makes decryption fail
AAD	the secret's ID	Ties each ciphertext to its own row
Key	**32 bytes** from `VAULT_MASTER_KEY`	Never stored in the database, never committed to git

2.3 Key and IV management

- **Key:** loaded from the environment and validated at startup.
- An invalid key stops the server from starting.

- A missing key means a random key is generated, with a warning (the handout FAQ allows this), but secrets are then lost on restart.
- `.env` is git-ignored; only `.env.example`, with an empty key, is committed.
- **IV:** a new random 12-byte value for every secret. Tests prove 1,000 encryptions give 1,000 different IVs, and the live database shows *distinct IVs = rows*.

Why not Base64, rot13 or plain hashing? Base64 and rot13 are encodings, not encryption: anyone can reverse them without a key. Hashing (MD5/SHA-256) is one-way, so the recipient could never get the secret back. A fixed IV or an unauthenticated mode (CBC/CTR without a MAC) would let attackers read patterns or tamper undetected. AES-GCM with a random IV avoids all of these.

2.4 What "zero trace" means here (stated precisely)

Level	Guarantee
✅ **Guaranteed**	The readable secret is **never written to disk** by the application. The database and its WAL file only ever hold ciphertext, IV and tag. Destroyed rows are removed with `DELETE`.
✅ **Hardening**	`PRAGMA secure_delete = ON` zeroes deleted data inside the database file, and the WAL is truncated after each sweep.
⚠️ **Not claimed**	Forensic erasure below SQLite: filesystem journals, SSD wear levelling, backups and OS swap may keep old bytes. Anything left there is encrypted.

2.5 Other defences

- Invalid input → 400, and the submitted value is **never echoed back**.
- Request bodies are limited to 128 KB (413).
- Strict Content Security Policy with no inline scripts.
- Clickjacking blocked (`X-Frame-Options: DENY`).
- API docs pages are turned off.
- Access logs are off (URLs contain IDs); application logs never contain secrets, request bodies or full IDs.

---

3. The Scraper Problem

> **In plain English:** when you paste a link into Slack, WhatsApp or Teams, their servers visit it immediately to build a preview card. If visiting the link destroyed the secret, the bot would "read" it first and your colleague would find a dead link. So **visiting** the link is always harmless. Only a person **clicking the button** reveals and destroys the secret.

3.1 How we stop it: layered defence

Layer	What it does	Stops
1. Look vs. act (the real guarantee)	`GET /view/{id}` only reads metadata. Revealing requires `POST /api/secret/{id}/burn`, which only the button sends.	All link previewers, which only ever send `GET`. Browser prefetch is also `GET`.

Layer	What it does	Stops
2. Bot detection	User-agent check on **every** route, before any database access: `bot`, `crawl`, `spider`, `Slackbot`, `facebookexternalhit` (handout minimum), plus Discord, WhatsApp, Telegram, Twitter, LinkedIn, `preview` and more	Bots that identify themselves
3. Human action	The reveal button runs JavaScript and asks for confirmation first	Fetchers that don't run JavaScript
4. Headers	`noindex`, `no-store`, `no-referrer`	Search indexing, caching, link leakage

What a bot receives: on `/view/*`, a generic "Secure secret" page with no ID, expiry or view count, which matches handout Scenario B's "HTML shell". On `/api/*`, a 403 JSON error. Anywhere else, an empty 403. **In every case, nothing changes in the database.**

`curl`, normal browsers and our CLI are **not** blocked; there are tests for that too.

Honest note: layer 2 can be fooled by a bot that pretends to be a browser. That's why layer 1 exists: a fake "browser" still can't burn anything with a `GET`.

---

4. Concurrency Strategy

> **In plain English:** imagine a concert ticket that can be scanned only once, with 20 people scanning copies at the same instant. The database acts as a single turnstile. It lets exactly one scan through, and at that moment the ticket is marked used. Everyone else is told "already used". There is no gap between *checking* the ticket and *using* it, so two people can't both get in.

4.1 The rule we guarantee

> **Only one request can successfully use up the final view of a secret.**

4.2 How: one atomic database operation

```
BEGIN IMMEDIATE;              -- take the database's write lock (one writer at a time)
UPDATE secrets
   SET views_remaining = views_remaining - 1
 WHERE id = ? AND views_remaining > 0 AND expires_at > ?     -- check AND decrement in ONE statement
RETURNING ciphertext, iv, auth_tag, views_remaining;
DELETE FROM secrets WHERE id = ?;   -- if that was the last view: same transaction
COMMIT;
-- decrypt afterwards, using only the data this request received
```

Why this is safe:

1. **Check and use are one statement.** There's no "read first, then update" gap, which is the classic double-read bug the handout warns about.

2. **SQLite lets only one writer in at a time**, across threads *and* processes. The other 19 requests wait their turn. When they run, `views_remaining` is already 0, or the row is gone, so their update matches nothing and they get 404.
3. **The final delete happens inside the same transaction**, so no one ever sees a half-finished state.
4. **No Python locks are used or needed.** The database enforces the rule, so it still holds with several server processes.

4.3 Proof that the test really catches the bug (mutation test)

A test that always passes proves nothing. So we temporarily swapped in the handout's **broken** pattern (read, then update) and fired the same 20 simultaneous requests:

Implementation	Requests that received the secret
Broken "read then update"	**4**, then **7**: leaked to multiple people ❌
Our atomic version	**exactly 1** in every run ✅

---

5. Results & Benchmarks

Automated test suite: 206 tests, 206 passing (≈38 s). The concurrency and cleanup tests run against a **real server process**, because an in-process test client handles requests one at a time and would hide race conditions.

5.1 The four results the handout asks for

Test	What we did	Expected	Result
Happy path (1-view burn)	Create (5-min TTL, 1 view) → reveal → reveal again → check DB	201 → 200 with secret → 404; row deleted	✅ exactly as expected; row count 0
Race condition (20 parallel)	20 simultaneous reveals of a 1-view secret, **repeated 10 times**	exactly 1×200, 19×404, row deleted	✅ **10/10 rounds**; `scripts/race.sh` prints `1 200 / 19 404 / 0 rows`
TTL expiry cleanup	Secret with 1 s TTL, never opened; real sweeper running	row physically deleted automatically	✅ gone within 2 sweeper cycles; reveal → 404
Tampered cipher payload	Edited ciphertext, IV or tag in the DB (flip, truncate, empty, swap rows)	clean 404, no stack trace, row destroyed	✅ **8/8 cases**

5.2 Full evidence

Area	What was verified	Result
Encryption at rest	Disk scan of `vault.db` + WAL for a known secret; IV/tag lengths; distinct IVs	✅ `0` matches · `12|16` bytes · distinct IVs = rows · WAL mode
Race, multi-view	20 parallel reveals of 2-, 3- and 5-view secrets	✅ exactly N winners, each view handed out once
Race, 4 server processes	`scripts/race.sh` against 4 uvicorn workers, 5 runs	✅ 5/5 runs `1 200 / 19 404 / 0`

Area	What was verified	Result
Scraper shield	12 real bot user-agents (Slackbot, facebookexternalhit, Twitterbot, Discordbot, WhatsApp, Telegram, LinkedIn, Googlebot, Bingbot…)	✅ shell page or 403; views unchanged; the human can still reveal
Not over-blocking	curl, Chrome, Firefox, Safari, httpx, vault-cli	✅ never blocked
Landing page is read-only	Viewed 10×	✅ views unchanged
Expiry edge cases	Expired but not yet swept · expired while the server was down	✅ unreadable · removed at startup
Hostile input	Oversized and chunked bodies, bad JSON, wrong types, path traversal, SQL-looking IDs, Unicode/NUL, `<script>` secrets	✅ 400/404/413; exact round trip; never rendered as HTML
Error hygiene	Forced internal errors on API and page routes	✅ generic JSON / generic HTML; no traceback, no secret
CLI	`echo … | ./cli/vault-cli`	✅ prints a working link
Real browser	Create → open link → reveal → reload	✅ works; no console or CSP errors

Tests per file: scraper 46 · API 31 · crypto 29 · password 29 · hostile input 25 · race 13 · end-to-end 11 · fingerprint 9 · tamper 8 · sweeper 5 = **206**.

5.3 Pre-freeze checklist (handout Part 10)

Item	Status
Database contains zero plaintext after ingestion	✅
Every encrypted record has a distinct IV	✅
`GET /view/:id` doesn't decrement or delete	✅
Crawlers (Slackbot, Twitterbot) can't consume the secret	✅
20 simultaneous requests → exactly one 200 and nineteen 404	✅
Background sweeper deletes expired records automatically	✅
Corrupted ciphertext → clean 404 without stack traces	✅
`cat secret.txt | ./vault-cli` works as documented	✅
REPORT.md committed	⏳ at submission
All code pushed to the target repository	⏳ at submission

5.4 Stretch goals (handout Part 7.2): all four built

All four are **optional and off unless chosen**, so the mandatory behaviour above is unchanged.

Stretch	What it adds	Key evidence
S1 Audit fingerprint (`VAULT_FINGERPRINT=1`)	The sender gets a SHA-256 fingerprint of the secret; the recipient's page computes the same, so both can confirm it arrived intact. **Never stored.**	9 tests; the browser recipient's hash matched the sender's (`sha256:1124b201…865b9a`)
S2 Password protection	Optional password. Only a salted scrypt hash is stored. A wrong or missing password → 401 **without using up the view**. 5 wrong tries destroy the secret.	29 tests incl. 20 parallel correct and wrong attempts; browser: wrong password left `views_remaining = 1`
S3 End-to-end encryption	The browser or CLI encrypts **before** sending; the key travels in the link after `#`, which browsers never send to the server. The server never sees the secret or the key.	11 tests; the server stored 76 bytes of ciphertext for a 27-byte secret; missing-key links can't burn
S4 Performance benchmark	`bench/bench.js` (autocannon) plus two optimisations	Target met with 4 workers (below)

5.5 Performance benchmark (stretch S4)

Target: more than 1,500 reads/second with 99% of requests answered in under 15 ms ("p99 < 15 ms"). "Read" means `GET /view/{id}`. **Setup:** autocannon 8, 15 s per scenario after a 3 s warm-up. Windows 11, Ryzen 5 5600H (12 threads). **The load generator ran on the same laptop**, sharing its CPU.

Two optimisations (all 206 tests still pass afterwards):

1. **Security middleware rewritten in plain ASGI.** The convenience wrapper (`BaseHTTPMiddleware`) halved throughput in a bare-app test (2,508 → 1,238 req/s).
2. **Read-only routes made `async`.** This skips a thread hop that cost about 40% (2,508 → 1,451 req/s). Their only database work is a microsecond lookup. Writes stay on threads, because they can wait for the database lock.

Setup	Reads/s	p99 latency	Target
1 worker, before optimisation	557	29 ms	❌
1 worker (recommended default)	**1,004**	**13 ms**	❌ speed · ✅ latency
4 workers	**2,531**	**12 ms**	✅ **met**
4 workers, 50 connections	2,758	68 ms	❌ latency (requests queue)

Additional numbers:

- Across repeated 4-worker runs, 10-connection results ranged from 2,531 to 2,989 reads/s, with p99 of 7–12 ms.
- Writes: about 500–900 creates or burns per second.
- Each burn run consumed all 3,000 test secrets **exactly once**, with no errors.

---

6. Limitations & Next Steps (honest account)

6.1 Known limitations

Limitation	Why it matters	Next step
No rate limiting	ID guessing is infeasible, but floods aren't throttled	Per-IP rate limits
Server holds the key for non-E2E secrets	Someone with the DB **and** the key could decrypt unread secrets	Use S3 end-to-end mode, or add key rotation and an HSM/KMS
Bot detection can be fooled	A bot pretending to be a browser passes the user-agent check	Already covered by layer 1 (GET never burns)
No forensic erasure below SQLite	SSDs and backups may keep encrypted remnants	Full-disk encryption on the host
Single host only	SQLite allows one writer at a time; not horizontally scalable	Postgres with the same atomic `UPDATE … RETURNING`
Crash between commit and reply	That view is used up without being delivered (fails closed)	Acceptable for one-time secrets
Writes have long tails	p99 130–180 ms, max 2–4 s under heavy contention (single writer)	Batch writes, or a server database
Benchmark on one Windows laptop	The load generator shared the CPU; Linux wasn't measured	Re-run on a Linux host

Stretch-feature limits:

- **E2E:** the link *is* the key, so a leaked full link can be decrypted. Pair it with a password sent separately.
- **E2E:** a wrong but well-formed key can't be detected before burning.
- **E2E:** needs HTTPS or localhost, and relies on the server's JavaScript being trustworthy.
- **Password:** each check costs about 455 ms of CPU (scrypt).
- **Password:** anyone with the link can destroy a protected secret with 5 wrong guesses.
- **Password:** it's a gate, not part of the encryption key.

6.2 Decisions we made where the handout was silent

Topic	Our choice	Reason
Unknown ID on `/view/{id}`	404 HTML page	The handout only defines the success case
Tampered data	404 and delete the row	The handout allows 404 or 400; 404 reveals nothing
`burned` field	`true` only when the last view is used	Makes multi-view secrets meaningful
Bot response	HTML shell on `/view`, 403 elsewhere	"Rejected or blank", as the handout allows
Invalid input	400 (FastAPI's default is 422)	Consistent with the handout's status codes
Recommended deployment	1 worker, single host	Simplest reliable setup; see the Windows note below

6.3 Problems we found during development (and fixed)

We tested beyond the happy path, and these are the real issues it uncovered:

Found	Cause	Fix
4-worker runs occasionally froze one worker for 10 s	A `faulthandler` stack dump showed uvicorn's multi-worker mode on **Windows** blocking in `socket.accept()`. It's a platform issue, not our code.	Documented: use 1 worker on Windows, multiple only on Linux. Correctness is unaffected.
CLI password prompt hung forever in Git Bash	Windows `getpass` waits on a console that doesn't exist there	Prompt only when a real terminal exists; otherwise a clear `VAULT_PASSWORD` hint
E2E link pasted into an open tab didn't enable Reveal	Changing only the `#fragment` doesn't reload the page	Re-read the key on `hashchange`
A "hidden" label stayed visible	A CSS `display` rule overrode the `hidden` attribute	A global `[hidden] { display: none !important }`
CLI failed under Git Bash	`python3` resolved to the Windows Store stub	A launcher that tries `python3`, then `python`

---

7. Quickstart

Requirements: Python 3.11+, SQLite 3.35+ (bundled with Python), and optionally Node 18+ for the benchmark.

```
# 1. install
python -m venv .venv
source .venv/Scripts/activate            # Windows Git Bash   (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt

# 2. configure a master key (64 hex chars)
cp .env.example .env
python -c "import secrets;print(secrets.token_hex(32))"     # paste the output as VAULT_MASTER_KEY in .env

# 3. run
uvicorn app.main:app --port 3000 --no-access-log           # then open http://localhost:3000
```

Try a secret with curl:

```
curl -s -X POST localhost:3000/api/secret -H 'Content-Type: application/json' \
     -d '{"secret":"my-database-password-xyz","ttl_seconds":300,"max_views":1}'
# -> 201 {"id":"…","view_url":"http://localhost:3000/view/…","expires_at":"…","views_remaining":1}

curl -s -X POST localhost:3000/api/secret/<id>/burn        # -> 200 {"secret":"my-database-password-xyz",…}
curl -s -X POST localhost:3000/api/secret/<id>/burn        # -> 404 (already destroyed)
```

From the terminal:

```
cat secret.txt | ./cli/vault-cli --ttl 300                 # prints the one-time link
```

Verify the claims in this report:

```
pytest                                                  # 206 tests
scripts/race.sh                                         # expect: 1 200 / 19 404 / 0 rows left
scripts/inspect_db.sh vault.db my-database-password-xyz # expect: distinct IVs, 12|16 bytes, 0 plaintext matches
cd bench && npm install && node bench.js                # performance benchmark (use a throwaway VAULT_DB_PATH)
```

Glossary (for non-technical readers)

Term	Meaning
AES-256-GCM	A widely trusted encryption method. "256" is the key size; "GCM" adds a seal that detects tampering.
IV (initialization vector)	A random value used once per encryption, so the same secret never encrypts to the same output twice.
Auth tag	The tamper-evident seal. If stored data is altered, decryption refuses.
Ciphertext / plaintext	The scrambled version of a secret, and the readable original.
TTL (time to live)	How long a secret stays available before it's automatically deleted.
Burn	Reveal the secret and destroy it in the same step.
Race condition	A bug where two simultaneous actions both succeed when only one should.
Atomic operation	A step that happens completely or not at all, with no in-between moment.
Link unfurler / crawler	A chat app's bot that visits links to show previews.
WAL (write-ahead log)	A SQLite mode that lets reads and writes happen at the same time safely.
p99 latency	The time within which 99% of requests are answered.
E2E (end-to-end) encryption	The secret is encrypted on the sender's device and decrypted on the recipient's, so the server never sees it.
scrypt	A deliberately slow password-hashing method that makes guessing stolen hashes expensive.
Worker	A copy of the server process; more workers can serve more requests at once.